PROJECT REPORT AND SUBMISSION PLAN

ReverseShield

A local static-analysis workbench for reverse-engineering exposure and demonstration-grade protection

Document	Value
Version	1.0 — final draft for submission
Date	7 November 2026
Platform	Fedora 43 (x86-64); Ubuntu/Kali/Windows as future work
Analysis mode	Static only; selected targets are never executed
Protection scope	Bundled samples for source profiles; binary metadata hygiene on one selected ELF copy
Companion document	ReverseShield_3-Day_Execution_Plan.docx

How to read this document

This is an execution document, not a marketing description. Every task has an output and an acceptance check. The project remains a static-analysis workbench; protected samples are compiled but never executed. Security claims are limited to observed artifact changes.

Document control

Item	Value
Title	ReverseShield: A Local Static-Analysis Workbench for Reverse-Engineering Exposure and Demonstration-Grade Protection
Document type	Project report and submission plan
Version / date	1.0 / 7 November 2026
Status	Final draft; security-analysis results marked as baseline or planned
Scope	Local static analysis, sample-only source protection, binary metadata hygiene, local DAST, Fedora packaging
Ethics	Only benign, self-authored samples or files the user is authorized to inspect; no target execution
Related document	ReverseShield_3-Day_Execution_Plan.docx (task-level schedule and checklists)

Claim boundary

ReverseShield is **not** a protection product, obfuscator, malware sandbox, or reverse-engineering-resistance benchmark. It is a measurement workbench. The demo transformation is intentionally weak and is reported with its weaknesses visible.

Abstract

Software is often distributed as compiled binaries that still carry symbols, debug information, source paths, and readable constants. These artifacts lower the cost of reverse engineering and can expose implementation details, internal naming, and policy logic. ReverseShield is a local, static-only application-security workbench that helps a user discover real installed applications, select ELF targets, run bounded static-analysis tools (including Ghidra headless), observe live progress, and compare a binary before and after a transformation applied to a copy of a bundled, self-authored sample. A separate binary-only profile removes symbols, debug information, and compiler notes from a copy of any selected ELF; it is metadata reduction, not code obfuscation.

The project contributes an integrated measurement pipeline rather than a new obfuscator. It demonstrates that symbol removal changes the metadata surface but leaves the logic recoverable: in a Level 5 policy sample, GNU `strip --strip-unneeded` reduced the file from 16,088 to 10,784 bytes and removed the symbol table and DWARF, yet Ghidra still recovered the policy function and its 2/3/99 resource mapping. It also demonstrates that a naive constant-blinding pass is not enough: only two integer literals were blinded, the key remained twice in `.data`, debug symbols were retained, and Ghidra removed the opaque guard as unreachable. The report argues that aggregate artifact counts are misleading and that content-level metrics, sanitized reporting, and explicit limitations are required for credible protection evidence.

The submission plan converts the prototype into a Fedora one-command bundle with a one-click demo, a before/after dashboard, a release build with no debug symbols or local paths, SAST/DAST evidence, and report redaction. The workbench remains static-only: it never executes a selected target, uploaded file, installed application, or protected build.

Contents

Page numbers refer to this PDF build. The Word file uses the same section order.

Section	Page
Document control	02
Abstract	02
1. Introduction	04
2. Background and related work	05
3. Threat model	06
4. Requirements and design principles	07
5. Architecture	08
6. Implementation	09
7. Evaluation	11
8. Security analysis	16
9. Packaging and distribution	19
10. Three-day execution plan	20
11. Discussion	20
12. Future work	21
13. Conclusion	21
References	21
Appendix A — API routes	22
Appendix B — Run storage	22
Appendix C — Bundled samples	22
Appendix D — Tool versions and commands	23
Appendix E — Report excerpts	23
Appendix F — Security scan commands	23
Appendix G — Glossary	24

1. Introduction

1.1 Problem context

Reverse engineering is a normal part of security assessment, interoperability work, and malware analysis. The cost of understanding a binary depends heavily on what the build leaks: function and file names, DWARF line tables, source paths, compiler flags, readable policy constants, and diagnostic strings. A binary that is 'stripped' is not necessarily protected; it is only less annotated. A binary that is 'obfuscated' is not necessarily resistant; it may only be more expensive to read. The distinction matters for developers who need to make honest claims about what a release build reveals.

Learners also need a safe way to practice. Running unknown binaries is risky and often prohibited. A local workbench that treats targets as data, bounds every tool call, and never executes the target gives a defensible environment for studying discovery, disassembly, decompilation, and artifact comparison.

1.2 Objectives

- Discover real installed applications and explicit local inputs without launching them.
- Let the user choose one or more ELF targets and run compatible static tools with live events.
- Produce structured evidence: metadata, sections, symbols, strings, disassembly, pseudocode, CFGs, and call edges.
- Compare a bundled sample before and after a source transformation, and apply binary metadata hygiene to a copy of any selected ELF.
- Demonstrate, rather than overclaim, the difference between metadata removal and protection.
- Provide a downloadable Fedora application that a reviewer can install and run without root.

1.3 Contributions

Contribution	What is new in this project	Evidence
Integrated local pipeline	Discovery → target selection → bounded static tools → live events → comparison → export	Working UI and API; 54 tests
Ghidra headless integration	Project-local JDK/Ghidra, temporary project, post-script export of functions, pseudocode, CFG and call edges	Level 5 run: 13 functions, 10 pseudocode bodies, 6 call edges
Honest protection comparison	Strip and demo constant blinding on sample copies, with content-level analysis of what survived	Two Level 5 reports with SHA-256 and metrics
Report safety	Design for sanitized evidence export (paths, key, PII, tokens) instead of sharing raw logs	Planned Day 1 task; baseline reports show the risk
Submission packaging	Fedora one-command install/run design with offline and lite options	Planned Day 1–3; explicit acceptance tests

1.4 Scope and constraints

In scope	Out of scope
Linux ELF analysis; Flatpak, RPM payload, folder, single ELF, upload	Windows PE, macOS Mach-O, Android APK, DEB
Static tools and Ghidra headless	Dynamic execution, emulation, ptrace, Frida, sandbox detonation
Source protection on bundled samples; binary metadata hygiene on one selected ELF copy	Binary code rewriting, packing, or protecting third-party applications
Local DAST against the workbench	Scanning external applications or networks
Fedora 43 packaging	Cross-platform installers in this window

2. Background and related work

2.1 Static analysis and the reverse-engineering workflow

A typical static workflow starts with file identification and ELF headers, then sections and symbols, then strings and disassembly, then decompilation and control-flow reconstruction. Each step answers a different question. Symbols tell you what the build claims; DWARF tells you where the source was and how lines map; strings reveal messages and paths; disassembly reveals instructions; a decompiler proposes a higher-level approximation. None of these steps proves behavior, and a decompiler's output is not the original source.

2.2 Symbol removal is not protection

GNU `strip` removes symbol and relocation information that is not needed for execution. It raises the cost of navigation but does not change the instructions that implement the logic. In the Level 5 experiment, stripping removed `.symtab` and DWARF and reduced the file by about one third, but Ghidra still found the policy function at a new address and recovered the same comparisons. This is the expected result; presenting strip as 'protection' would be misleading.

2.3 Constant blinding and opaque predicates

Constant blinding replaces a literal with an expression that evaluates to the same value at runtime, ideally in a form the compiler cannot fold. A volatile key stored in memory can prevent constant folding, but it does not prevent a human or a decompiler from reading the key and simplifying the expression. An opaque predicate is a branch whose condition is always true or false but is intended to look meaningful. If the predicate is `x ^ x`, a decompiler can prove it is zero; the branch adds bytes, not analytical cost. These techniques are useful as teaching examples precisely because their limits are measurable.

2.4 SAST, DAST, and data leakage

SAST reviews source or bytecode for dangerous patterns; DAST exercises a running application over HTTP. For a local security tool, both are relevant: SAST can find subprocess, path, and file-handling risks in the workbench itself; DAST can test the loopback API, token enforcement, upload limits, origin checks, and error handling. Separately, the reports produced by a security tool are themselves sensitive: they can contain absolute paths, usernames, command lines, keys, and fragments of the target. Data-leak prevention is therefore part of the product, not an afterthought.

Relevant weaknesses include CWE-200 (exposure of sensitive information), CWE-798 (hard-coded credentials), and CWE-312 (cleartext storage of sensitive information). The demo key is intentionally disclosed for reproducibility, but it must be labelled and removed from shareable exports.

2.5 Gap addressed by ReverseShield

Existing tools are strong individually: binutils for metadata and disassembly, Ghidra for decompilation, ZAP for DAST, Semgrep/Bandit for SAST. What is missing in a student project is an integrated, local, bounded pipeline that connects these tools to a reproducible before/after comparison and forces the author to state what the evidence does and does not show. ReverseShield fills that gap at demonstration scale.

3. Threat model

3.1 Assets

Asset	Why it matters	Exposure
Selected binaries and snapshots	May be proprietary or sensitive even when benign	`runs/<job>/original-N.elf` (0600)
Reports and logs	Contain paths, symbols, strings, pseudocode, commands	`report.json`, `events.jsonl`
Demo key and transformation source	Enables reversal of the demo blinding	`.data`, `report.json`, `protected/`
Host filesystem	Tool arguments and discovery paths touch local files	Read-only analysis; snapshots
User identity	Home paths and usernames can leak into reports	Commands, DWARF, logs

3.2 Actors and trust boundaries

Actor / boundary	Capability	Control
Local user	Starts jobs, selects targets, downloads reports	Loopback + per-session token + authorization checkbox
Browser	Same-origin UI; no cross-origin API	Origin check, CSP, no CORS
Uploaded file	Untrusted bytes parsed by tools	Magic check, 64 MiB cap, snapshot, never executed
Tool subprocess	Reads the snapshot and writes artifacts	Argv lists, `shell=False`, timeout, output caps, isolated workdir
Report recipient	Reads exported evidence	Sanitized export by default; raw report internal
Network attacker	Cannot reach loopback by default	Bind 127.0.0.1; no remote deployment

3.3 STRIDE summary

Threat	Example	Current / planned control
Spoofing	Foreign page calls the API	Session token, Origin check, same-origin UI
Tampering	Target changes between discovery and analysis	Device/inode/size/mtime identity check; snapshot copy
Repudiation	No record of what ran	Sequenced events, tool commands, hashes in report
Information disclosure	Reports leak paths, key, PII	Sanitizer; raw export warning; 0600 files
Denial of service	Huge file or tool output	64 MiB target cap, 1 MiB output cap, timeouts, bounded jobs
Elevation of privilege	Path traversal or command injection	Absolute-path validation; argv lists; basename-only upload names

3.4 Assumptions and residual risk

- The host is trusted; a local attacker who can read `runs/` can read the evidence.
- Ghidra and binutils parse untrusted bytes and may have their own vulnerabilities; isolation is a future requirement for hostile inputs.
- The workbench is not a malware sandbox; loopback and snapshots are not a detonation environment.

- The demo protection is not a security boundary; the key is recoverable and the report discloses it by design.
- No dynamic behavior is observed, so functional equivalence of transformed samples is not verified.

4. Requirements and design principles

4.1 Functional requirements

ID	Requirement	Acceptance
FR-01	Discover installed Flatpak applications and list ELF targets	9 apps listed on the test host
FR-02	Discover a folder, single ELF, RPM payload, or uploaded ELF/RPM	Targets listed with size and analyzability
FR-03	Run selected static tools with live events	Per-tool start/end/log/artifact events
FR-04	Run Ghidra headless and export functions, pseudocode, CFG, call edges	Level 5: 13 functions, 10 bodies
FR-05	Create before/after reports for source-sample and binary-metadata transformations	Strip, demo-blinding and binary-hygiene reports with hashes
FR-06	Compare artifacts and label missing metrics as N/A	No false zero when a provider was not selected
FR-07	Persist reports and artifacts under `runs/<job_id>/`	Files survive server restart
FR-08	Export report and artifact from the UI	Download works; sanitized export planned
FR-09	Install and run on Fedora without root	Planned `install.sh` / `run.sh`
FR-10	Refuse unauthorized or unsupported inputs	415/413/403 paths tested

4.2 Non-functional requirements

ID	Requirement	Design response
NFR-01	Local-first and offline-capable	Loopback bind; project-local tools; no telemetry
NFR-02	Static only	No exec/emulation; target is data; protected builds are never run
NFR-03	Bounded resources	Timeouts, output caps, job limits, 64 MiB target cap
NFR-04	Reproducible evidence	SHA-256, tool versions, commands, event sequence
NFR-05	Honest labels	`demonstration_only`, `behavior_verified: false`, no security score
NFR-06	Usable by a beginner	One-click demo; clear errors; docs; no root
NFR-07	Testable	Unit/API/UI-contract tests; opt-in Ghidra test
NFR-08	Safe to share	Sanitized reports; secret/PII checks; no raw paths

4.3 Design principles

- **Evidence over adjectives.** Every claim maps to a file, hash, count, or screenshot.
- **Static by default.** Execution is a separate, future, explicitly authorized mode.
- **Least privilege.** Loopback, token, 0600 snapshots, 0700 job directories, no shell.

- **Bounded everything.** If a tool can run forever, it will; cap time, bytes, jobs, and targets.
- **Show the weakness.** A comparison that hides the key or debug symbols is not evidence.

5. Architecture

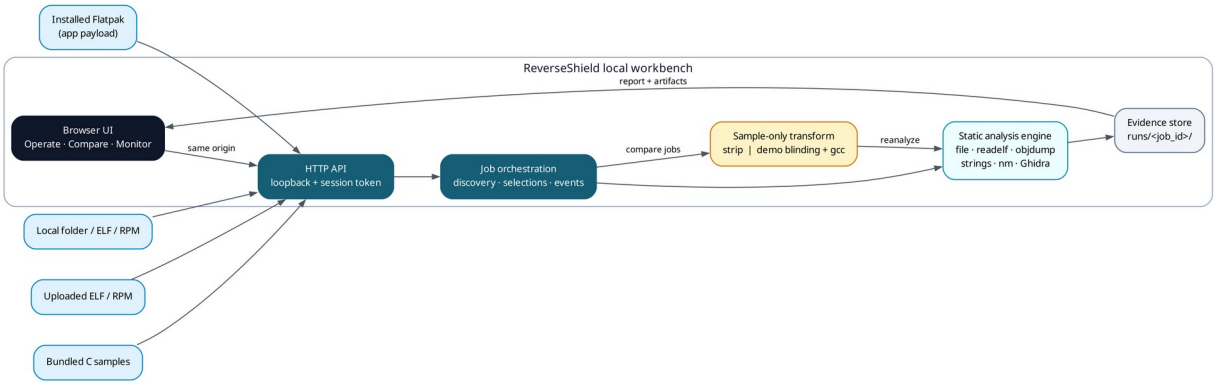


Figure 1. High-level architecture: inputs, local API, job orchestration, static engine, sample-only source transform, binary metadata hygiene, and evidence store.

5.1 Components

Component	Responsibility	Key files
UI	Operate, Compare, Monitor, Read Me; live events; downloads	`static/index.html`, `static/workbench.js`, `static/workbench.css`
HTTP API	Loopback server, token/origin checks, JSON routes, upload	`app.py`, `http_upgrade.py`
Discovery	Flatpak list/targets, folder scan, RPM payload, path/ELF	`discovery.py`, `rpm_ingest.py`
Job orchestration	Selections, async jobs, events, cancellation, report persistence	`workflow.py`
Analysis engine	Bounded tool adapters, parsers, Ghidra export	`engine.py`, `analysis.py`, `ghidra_scripts/`
Protection	GNU strip; demo constant blinding + guard; binary metadata hygiene	`demo_protection.py`, `binary_protection.py`
Storage	Snapshots, logs, reports, artifacts	`runs/<job_id>/`

5.2 Data flow



Figure 2. Per-job data flow. Source profiles 4–6 are sample-only; binary metadata hygiene is available for one selected ELF copy.

5.3 Storage model

Path	Contents	Permissions
`runs/<job_id>/original-N.elf`	Snapshot of each selected target	0600
`runs/<job_id>/events.jsonl`	Sequenced live events	Job dir 0700
`runs/<job_id>/report.json`	Final report: events, results, artifacts, coverage, limitations	Job dir 0700

Path	Contents	Permissions
`runs/<job_id>/analysis-*/`	Ghidra export and logs for analyze jobs	Job dir 0700
`runs/<job_id>/before-*/`, `after-*/`	Ghidra export and logs for comparisons	Job dir 0700
`runs/<job_id>/symbols-removed.elf`	Strip artifact	0600
`runs/<job_id>/protected/`	Protected ELF and transformed sources	0600 / 0700
`runs/inputs/<hex>/`	Uploaded ELF/RPM and RPM payload	0700

5.4 Security controls

Control	Implementation	Limitation
Loopback bind	Server listens on 127.0.0.1	Local malware can still connect
Session token	Random per server run; `X-ReverseShield-Token`	Not a user account or RBAC
Origin check	Reject foreign `Origin`	Not a substitute for auth
Upload caps	64 MiB ELF, 256 MiB RPM	Resource use still possible within caps
No execution	Subprocess tools read bytes; no target launch	Parsers may have vulnerabilities
Snapshot identity	dev/inode/size/mtime check before and after copy	Race window is reduced, not zero
Report permissions	0600 files, 0700 dirs	Same-user processes can read

6. Implementation

6.1 Discovery

Discovery is read-only and never launches a discovered file. Flatpak applications are listed with `flatpak list`; the selected application's `files/` tree is scanned for ELF regular files. Local folders are scanned with bounds on entry count and size. RPM packages are unpacked as data with `rpm2cpio` and a bounded `cpio` extractor; scripts are never executed. Uploads are written to a run-local input workspace, magic-checked, and then discovered like a local file.

Each target gets a deterministic `target_id` from its relative path, plus size and an `analyzable` flag. The selection stores an identity tuple (device, inode, size, mtime); the snapshot step re-checks it before and after copying. If the file changes between discovery and analysis, the job stops rather than analyzing a different file.

6.2 Analysis engine

The engine runs each selected tool as an argv list with `shell=False`, a per-tool timeout, and a 1 MiB output cap. It parses structured artifacts from `readelf`, `objdump`, `nm`, `strings`, and Ghidra's JSON export. Every tool record contains the command, status, return code, duration, truncation flag, version, and output. Missing or failed tools are reported as such; they are not silently treated as zero.

```
# Ghidra headless invocation (bounded; project deleted after export)
analyzeHeadless <workdir>/ghidra_project ReverseShield \
  -import <snapshot> \
  -scriptPath ghidra_scripts \
  -postScript ReverseShieldExport.java <workdir>/ghidra-export.json \
  -deleteProject -analysisTimeoutPerFile 150 -max-cpu 2 \
  -loader-loadLibraries false \
  -log <workdir>/ghidra.log -scriptlog <workdir>/ghidra-script.log
```

Ghidra uses the project-local Temurin JDK 25 and Ghidra 12.2 under tools/; nothing is installed system-wide. The post-script exports functions, pseudocode, CFG nodes/edges, symbols, strings, call edges, and coverage limits. The temporary Ghidra project is deleted; only the JSON export and logs remain.

6.3 Workflow, events, and cancellation

A single-worker thread pool serializes analysis jobs to avoid tool contention. Jobs emit sequenced events (queued, phase, tool_start, log, tool_end, artifact, terminal state). Events are appended to events.jsonl and kept in memory up to a bound; the final report is written even when a job fails. Cancellation sets an event; the engine terminates the process group and marks the tool cancelled. A failed reanalysis preserves the baseline report and the protection state instead of losing both.

6.4 Comparison

Comparison is intentionally provider-aware. If no selected tool could collect a metric, the comparison shows N/A, not 0. The current summary counts are useful for a quick view but are not a resistance score. The report includes strings/symbols added and removed, but these lists contain metadata noise (debug section names, Ghidra labels). The submission upgrade adds content metrics and separates ELF symbol tables from Ghidra-recovered functions.

6.5 Symbol removal

For a bundled sample, the workflow copies the selected ELF to original-0.elf, analyzes it, runs strip --strip-unneeded -o symbols-removed.elf original-0.elf, records the hashes, reanalyzes the stripped copy, and compares the two reports. The original sample is never modified. The state is labelled symbol_removal_applied, with behavior_verified: false and the limitation that symbol removal is a baseline transformation, not code obfuscation.

6.6 Demo constant blinding

The demo transformer reads copies of the sample sources, inserts a static volatile uint32_t rs_key, replaces selected integer literals with XOR expressions, adds an opaque guard to integer-returning functions, copies headers, and compiles the result with GCC. The protected ELF is written under protected/, chmod 0600, and never executed. The state is labelled demo_obfuscation_applied, strength: demonstration_only, behavior_verified: false.

```
/* Illustrative transformed source (current implementation) */
static volatile uint32_t rs_key = 0x9E3779B9u;

int policy_allows(const struct access_request *request) {
   if ((rs_key ^ rs_key) != 0u) { return 0; }
   if (request == 0 || request->tenant_id == 0u ||
      !valid_role(request->role)) return 0;
   if ((request->flags & (0x1E3779B9u ^ rs_key)) != 0u) return 0;
   return (int)request->role >= resource_minimum_role(request->resource);
}
```

This implementation is deliberately simple and is the subject of the evaluation: it blinds 7u and 0x80000000u, but not the enum values 1/2/3 or the literal 99; it adds a guard that a decompiler can fold; it keeps debug symbols; and it uses a fixed key. Those are not hidden defects; they are the evidence that motivates the upgrade plan.

6.7 Binary metadata hygiene

A binary-only profile operates on a copy of one selected ELF, without source. It runs strip --strip-all to remove the static symbol table and debug information, then objcopy to remove .comment, .annobin.notes, .gnu.build.attributes and .note.gnu.build-id. The original file is never modified and the copy is chmod 0600. The state is labelled binary_metadata_hygiene_applied, strength: metadata_only, obfuscation: false, behavior_verified: false.

```
strip --strip-all binary-hygiene.elf
objcopy --remove-section=.comment binary-hygiene.elf
objcopy --remove-section=.annobin.notes binary-hygiene.elf
objcopy --remove-section=.gnu.build.attributes binary-hygiene.elf
objcopy --remove-section=.note.gnu.build-id binary-hygiene.elf
readelf -S binary-hygiene.elf | grep -E 'debug|symtab|comment|annobin' || echo "no metadata"
strings -a binary-hygiene.elf | grep -E '/home|transformed-src' || echo "no local paths"
```

> **What this is and is not**
>
> This is **binary metadata hygiene**: it reduces the annotation surface of any ELF copy. It does **not** rewrite instructions, encrypt code, add a runtime loader, or resist a determined reverse engineer. Binary code rewriting and packing remain future work.

6.8 User interface

The UI is a dark cyan, terminal-inspired single-page application. Operate handles source discovery, target preview, authorization, tool selection, and job start. Monitor shows live events and recent jobs. Compare shows protection state, warnings, a metadata/difference table, and download buttons. Read Me documents scope and limitations. The UI never claims a security score and labels static-only mode persistently.

6.9 Testing

The suite contains 54 tests (one opt-in Ghidra test skipped by default). It covers ELF metadata parsing, discovery, RPM ingestion, engine parsing, async workflow, comparison, demo protection, API orchestration, and UI contracts. The tests never execute a sample, target, or protected build. RUN_GHIDRA_TESTS=1 runs the real Ghidra smoke test.

```
python3 -m unittest discover -s tests
# Ran 54 tests ... OK (skipped=1)

RUN_GHIDRA_TESTS=1 python3 -m unittest discover -s tests -p 'test_engine.py' -v
```

7. Evaluation

7.1 Method and environment

Item	Value
Host	Fedora Workstation 43, x86-64, Wayland
Python	3.14.7
Compiler	GCC 15.3.1 (Red Hat 15.3.1-1)
Binutils	GNU readelf/objdump/nm/strings 2.45.1-5.fc43
Ghidra / JDK	Ghidra 12.2 / Temurin 25.0.5+8
Sample	Level 5 · Policy Engine (multi-file C, 16,088 bytes)

Item	Value
Constraint	No target execution, no emulation, no dynamic tracing

Two transformations were evaluated: GNU symbol removal and the demo constant-blinding transformer. Both operate on a run-local copy; the original sample is unchanged. The reports are stored under runs/<job_id>/report.json; the figures below are taken from the actual Level 5 runs, not estimated.

7.2 Symbol removal on Level 5

Metric	Before	After	Interpretation
File size	16,088 B	10,784 B (−33%)	Metadata and sections removed
Sections	36	27	Debug and symbol sections removed
`.symtab`	Present	Absent	No static symbol table
Debug info	Present	Not detected	No DWARF line/source map
`nm`	29 named entries	No symbols	No names for the analyst
Strings	132	61	Debug/path/symbol strings removed
Ghidra functions	13	11	Two functions not recognized
Pseudocode bodies	10	8	`policy_reason` lost as a function
Call edges	9	5	Fewer named edges, not less logic
BuildID	`25ed7523…`	`25ed7523…`	Still correlates with the original
Entry point	`0x400360`	`0x400360`	Unchanged
User messages	6	6	Policy strings remain

The important result is not the size reduction. Ghidra still recovered the policy logic at FUN_0040047e: the same tenant, role, and flag checks and the same 2/3/99 resource mapping. main became FUN_00400446 but still called the policy function with the same structure. policy_reason at 0x4004d4 was not recognized as a function after stripping, so one of the three application functions lost its decompilation; the other two remained analyzable. Symbol removal removed the names and the map, not the algorithm.

> **Finding 1**
> Strip is a metadata transformation. It is useful as a baseline and as a control in the experiment, but it is not protection. Presenting it as such would be contradicted by the Ghidra output in the same report.

7.3 Demo constant blinding on Level 5

Metric	Before	After	Interpretation
File size	16,088 B	16,528 B (+2.7%)	Guards and key add bytes
`.text`	443 B	581 B (+31%)	Real code growth
`policy_allows`	86 B	161 B (+87%)	Guard and XOR expansion
`policy_reason`	71 B	105 B (+48%)	Inlining/expansion
`main`	56 B	85 B (+52%)	Guard and tenant encoding
XOR instructions	3	6	Blinding and guard
`rs_key` references	0	13	Key is visible and used
CFG nodes	30	33	Small structural change

Metric	Before	After	Interpretation
ELF `.symtab`	36	38	Key symbols added
Ghidra functions	13	13	No reduction
Program strings	6	6	No message obfuscation
Ghidra time	6.38 s	6.34 s	No analyst cost

The aggregate picture is nearly flat. The content picture is worse for the protection claim: the two .data copies of rs_key contain b9 79 37 9e (0x9E3779B9) at 0x403004 and 0x403008; the report itself contains "key": "0x9E3779B9"; the build still says with debug_info, not stripped; and only two distinct literals were blinded (7u and 0x80000000u). The policy table values 1, 2, 3 and 99 remain in policy_allows as cmp $0x2, cmp $0x3, $0x1, and mov $0x63.

Item	Observed	Why it matters
Debug/symbols	`policy_allows`, `policy_reason`, `main`, `rs_key` still named	The analyst gets the map and the key symbol
Key storage	Two copies in `.data`, plus report field	Trivial to read and XOR back
Literal coverage	2 distinct values blinded; 1/2/3/99 remain	The policy table is still readable
Guard	`rs_key ^ rs_key`; Ghidra marks block unreachable	Adds bytes, not analysis work
Key uniqueness	Fixed 0x9E3779B9 across builds	Stable signature; no per-build diversity
Report size	282,138 B; 187 events	Contains paths, commands, logs, key

Finding 2

The demo transformer changes the artifact but not the analyst's conclusion. Ghidra recovers the same policy logic, including the resource mapping; the key is in the file and in the report; the guard is removed. This is a useful negative result, and it is the reason the upgrade plan focuses on coverage, artifact hygiene, and honest metrics.

7.4 Binary metadata hygiene on Level 5

The binary-hygiene profile was run on the Level 5 ELF as an external target copy. It used the same six tools as the other experiments. This is a metadata-only profile: the goal is to remove annotations from any selected ELF, not to change its instructions.

Metric	Before	After	Interpretation
File size	16,088 B	9,960 B (−38%)	Debug, symbol, comment and build-id data removed
Sections	36	23	Metadata sections removed
ELF `.symtab`	Present	Absent	No static symbol table
Debug info	Present	Not detected	No DWARF
Program strings	132	36	Debug/path/compiler strings removed; six policy messages remain
Ghidra functions	13	11	Two functions not recognized after stripping
Pseudocode bodies	10	8	Same code, fewer names
Ghidra time	6.71 s	5.92 s	No analyst slowdown
Original file	`6c0be224…`	`6c0be224…`	Unchanged; only the copy was modified

Metric	Before	After	Interpretation
Artifact mode	—	0600	Not executable; evidence only

The six policy messages and the policy logic remain. Ghidra still recovered the decision function, because removing symbols does not remove instructions. The result is a smaller, less annotated copy; it is not a code obfuscator. Binary instruction rewriting and runtime packing remain future work.

7.5 What Ghidra recovered

The protected policy_allows pseudocode still shows the role check, the flag check expressed through rs_key, and the 2/3/99 mapping. main still constructs the same request; only the tenant value is expressed as rs_key ^ 0x9e3779be. The following excerpts are shortened for readability.

```
/* After demo blinding, Ghidra 12.2 (shortened) */
if (request == NULL) return 0;
if (request->tenant_id != 0) {
  if (2 < request->role - ROLE_VIEWER) return 0;
  if (((rs_key ^ 0x1e3779b9) & request->flags) == 0) {
    iVar2 = 2;
    if (resource != RESOURCE_ARCHIVE) {
      iVar2 = 3;
      if (resource != RESOURCE_EXPORT) {
        iVar2 = 1;
        if (resource != RESOURCE_REPORT) iVar2 = 99;
      }
    }
    return (int)(iVar2 <= (int)request->role);
  }
}
return 0;
```

For comparison, the stripped binary's FUN_0040047e contains the same branches and constants. The decompiler does not need the original names to recover the decision logic; it needs the instructions, and those are unchanged by stripping.

7.6 Real application: EasyEffects

Observation	Value
Installed Flatpak applications listed	9
EasyEffects ELF targets discovered	11
Main executable size	13.3 MB
Main executable sections	32
Main executable symbols	911
Runtime dependency	`KDE6Platform` listed, not scanned
`objdump` output	Truncated by the bounded output limit
Ghidra	Not automatic for this app in the demo; runtime cost is high
Execution	None; the application was never launched

EasyEffects demonstrates the discovery path on a real installed application: the workbench lists its ELF files, lets the user choose targets, and analyzes the selected main executable with metadata tools. It also demonstrates the limits: a 13.3 MB binary with 911 symbols produces large output, and Ghidra should be a deliberate, optional run rather than a default for every target. The runtime dependency is listed but not scanned, by design.

7.7 UI and latency

On Level 5, the UI showed the full event stream (187 events) and Ghidra completed in 6.34 s. The analysis view exposes metadata, sections, functions, symbols, strings, pseudocode, CFG, call edges, raw output, and coverage limits. The comparison view shows the protection state, warnings, hashes, and a basic differences table. Figures 3 and 4 show the current Operate and Compare screens.

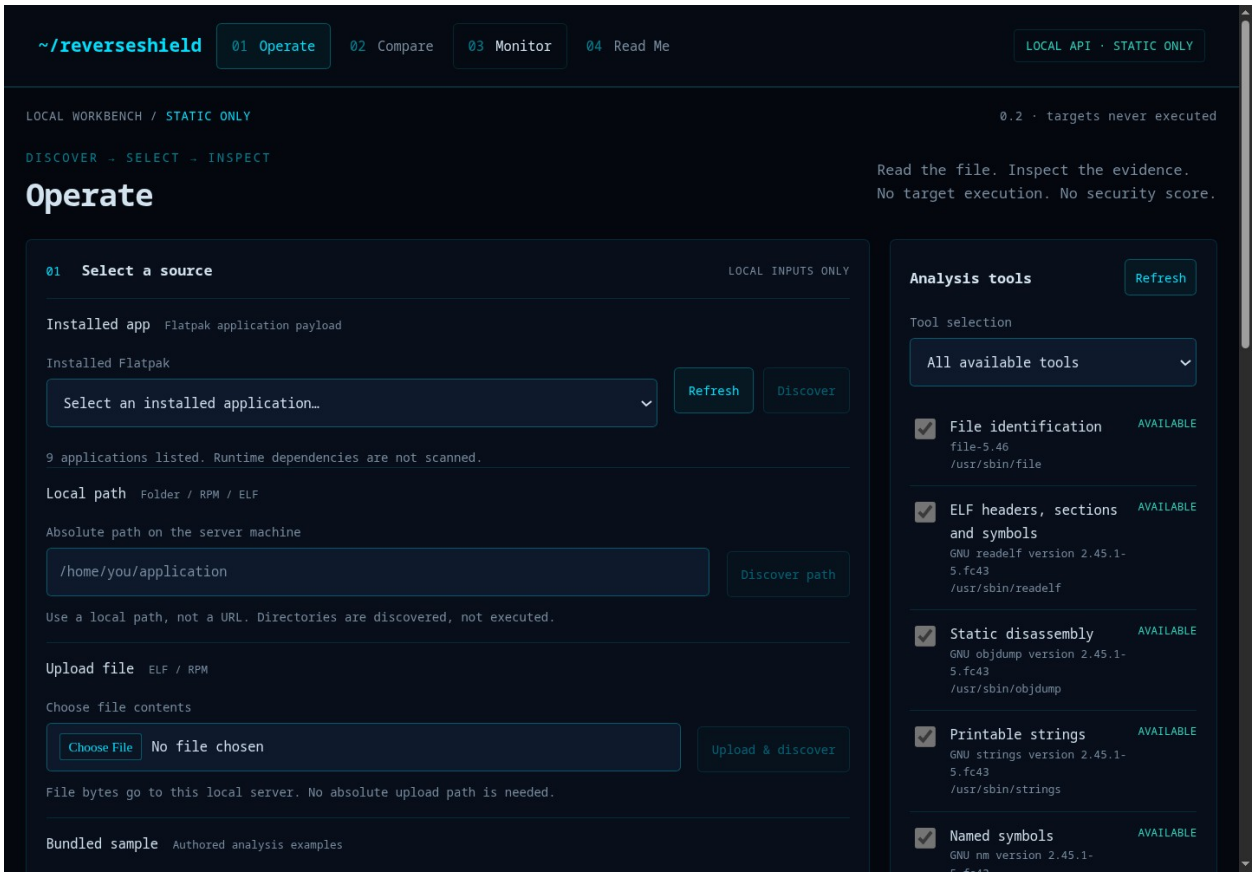


Figure 3. Operate screen: source selection, target preview, authorization, tools, and job controls.

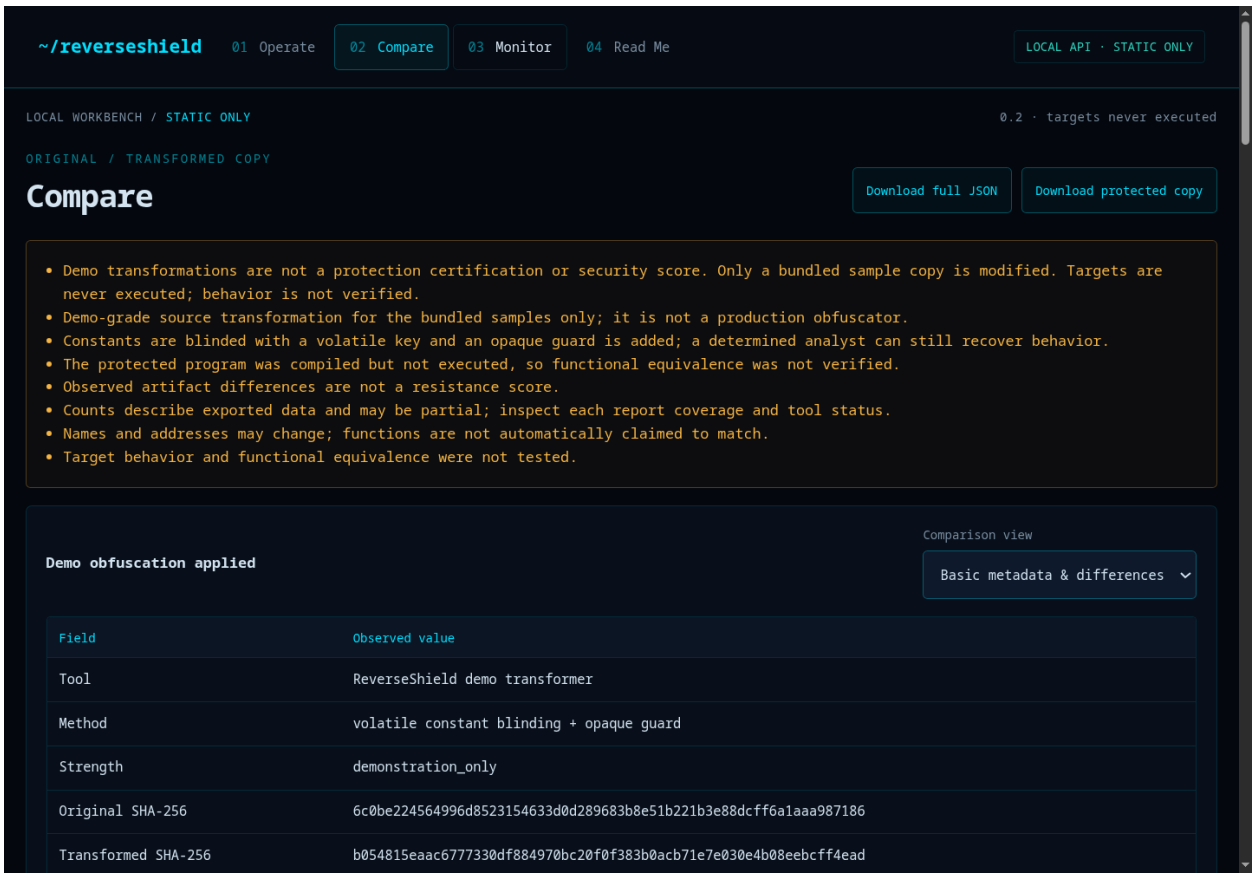


Figure 4. Compare screen: warnings, hashes, downloads, and the current metadata/differences table.

7.8 Threats to validity

- **Small sample.** Level 5 is 16 KB and self-authored; results do not generalize to large stripped production binaries.
- **One toolchain.** Ghidra 12.2, GCC 15.3.1 and binutils 2.45.1; other versions may produce different labels.
- **No behavior test.** The protected sample was compiled but not executed, so functional equivalence is unverified.
- **Decompiler approximation.** Pseudocode is not source and may omit indirect calls or runtime-loaded code.
- **Provider mixing.** Current symbol/string counts merge ELF and Ghidra artifacts; the upgrade separates them.
- **One host.** No cross-machine or cross-distribution reproducibility test yet.

8. Security analysis

8.1 SAST baseline

A preliminary SAST baseline was run on 7 November 2026 against the project Python and JavaScript files, excluding tools/, runs/, backups/ and generated bytecode. This is a starting point for the Day 2 review, not a claim that the code is vulnerability-free.

Tool	Version	Scope	Result
Bandit	1.9.4	11 Python modules, 1,965 LOC	18 low, 0 medium, 0 high
Semgrep	1.184.0	Python + JavaScript; security-audit, secrets, Python, JavaScript packs	0 findings
detect-secrets	1.5.0	Project files, excluding tools/runs/backups	0 findings

Bandit rule	Count	Meaning	Disposition
B404	8	Subprocess module import	Expected for a tool orchestrator; review call sites
B603	10	Subprocess call without shell	Expected; argv lists, `shell=False`, bounded timeouts
B602	0	Shell injection	No `shell=True` found
B607	0	Partial executable path	Tools resolved via `shutil.which` and paths validated

The zero medium/high result is encouraging but limited: Bandit does not model the HTTP API, path traversal, or report data leakage. Semgrep's zero result depends on the selected rule packs; project-specific rules and C sample rules are still needed. detect-secrets scanned the working tree, not the full Git history.

8.2 DAST plan: multiple methods

DAST is scoped to the local workbench at http://127.0.0.1:8765. The server binds loopback only. The ZAP baseline is the P0 method because it is passive and repeatable; the other methods are alternatives or stretch coverage. These scans were **not executed in this draft**; the table defines the procedure and expected evidence for Day 2.

Method	Command (example)	Coverage	Limits
ZAP baseline	`docker run --rm --network=host -v "$PWD/docs/security:/zap/wrk:Z" ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t http://127.0.0.1:8765 -r zap-baseline.html -J zap-baseline.json`	Passive rules, headers, cookies, spider	No authenticated state-changing requests; not a full pentest
ZAP API scan	`zap-api-scan.py -t openapi.json -f openapi`	Endpoint coverage, request/response	Needs a spec; P2 to write one
Nuclei	`nuclei -u http://127.0.0.1:8765 -severity low,medium,high,critical -o nuclei.txt`	Known templates, misconfig	Generic templates; not app-specific
Nikto	`docker run --rm --network=host sullo/nikto -h http://127.0.0.1:8765`	Server misconfig, files	Noisy; limited SPA coverage
Manual proxy	ZAP/Burp + browser; explore Operate/Compare/Monitor	Token, origin, upload flow	Manual checklist required
Custom API tests	`curl` matrix for 403/413/415/404/409	Business logic and validation	Only written cases

Scope and ethics

Only scan the local workbench you own. Do not use active scan mode against third-party sites. Stop the server after the scan. Keep raw ZAP output internal; include a sanitized summary and the exact command in the report.

8.3 API security controls and negative tests

Control / test	Expected	Why
Missing `X-ReverseShield-Token`	403	Prevents drive-by local calls
Foreign `Origin`	403	Blocks cross-site requests
`Transfer-Encoding: chunked`	400	Rejects unsupported body framing
Upload > 64 MiB	413	Bounds memory and disk
Non-ELF upload	415	Rejects unsupported formats early

Control / test	Expected	Why
`filename=../../etc/passwd`	basename only	Prevents path traversal
Unknown job ID	404	No job enumeration
Report before completion	409	No partial-file confusion

8.4 Data-leak prevention

The current reports are evidence-rich and share-hostile. A scan of the Level 5 strip report found **238 occurrences** of `<HOME_PATH>` across commands and logs; the report contains a demo key in protection.key, plus source paths, Ghidra logs, and CA paths. The submission must separate the internal raw report from the shareable sanitized report.

Data class	Examples	Sanitized form
Local identity	<HOME_PATH>, username, project root	`<HOME>`, `<USER>`, `<PROJECT>`
Commands	Absolute input/output paths	Keep tool/flags; replace path arguments
Demo key	`0x9E3779B9`	`<DEMO_KEY>` or omit; keep method/strength
PII	Email, non-loopback IP, names	`<EMAIL>`, `<IP>`
Secrets	Token-like strings, PEM blocks, cloud keys	`<REDACTED_SECRET>`
Evidence	SHA-256, sizes, versions, counts, status	Unchanged

The sanitizer is a recursive walk over the report object, not a regex over the serialized file. This preserves JSON structure and hashes. Tests must assert that the sanitized output contains no /home/, no username, no demo key, no email, no token pattern, and that running it twice is idempotent.

8.5 Findings and remediation

Finding	Severity	Remediation	Status
Subprocess import/call (B404/B603)	Low	Argv lists, `shell=False`, validated paths, timeouts; document why	Accepted with justification
Report contains absolute paths	Medium (privacy)	Sanitized export; UI warning; tests	Planned Day 1
Report contains demo key	Medium (demo secret)	Redact in sanitized export; label raw as internal	Planned Day 1
Debug symbols in protected build	Medium (protection evidence)	`-g0` + strip + section removal	Planned Day 1
Guard is folded by Ghidra	Low (demo quality)	Replace with runtime-dependent guard or label as syntactic	Planned P1/P2
Only 2 literals blinded	Medium (coverage)	Encode policy table; all literals	Planned Day 1

8.6 Limitations

- SAST baseline is preliminary; it does not include the final code or full Git history.
- DAST is planned, not executed in this draft; ZAP baseline is passive and will not exercise authenticated mutations.
- No dynamic binary analysis, exploitation, or fuzzing was performed.
- No third-party penetration test; the workbench is a local tool, not a hardened multi-user service.
- Report sanitation is a design; the current reports remain raw and must not be shared as-is.

9. Packaging and distribution

9.1 Requirements

- One download, one install command, one run command on Fedora 43.
- No root and no system-wide writes; project-local tools and state.
- Offline option for the committee; lite option for a small upload.
- A desktop entry that opens the local UI in the default browser.
- A clean uninstall that removes the entry and state, not the user's runs.

9.2 Options

Option	Effort	Size	Recommendation
Fedora tarball + `install.sh` + `run.sh`	Low–medium	Project + optional 1.2 GB tools	P0
Lite tarball with first-run tool fetch	Low	~2 MB + download	P0 alternative
Podman/Docker image	Medium	~1.2 GB	P1/P2
AppImage	High	50–100 MB + external Ghidra	P2
Windows `.exe`	High	Cross-platform tools	Future

9.3 Recommended bundle

```
ReverseShield-1.0-fedora/
 ├── app.py, analysis.py, workflow.py, engine.py, ...
 ├── static/
 ├── samples/
 ├── tools/              # optional: JDK 25 + Ghidra 12.2
 ├── docs/
 ├── install.sh
 ├── run.sh
 ├── uninstall.sh
 ├── reverseshield.desktop
 ├── VERSION
 ├── CHECKSUMS.txt
 └── README-FIRST.txt
```

9.4 Install, run, uninstall

```
tar -xzf ReverseShield-1.0-fedora.tar.gz
cd ReverseShield-1.0-fedora
./install.sh        # checks, venv, desktop entry, smoke test; no sudo
./run.sh            # starts 127.0.0.1:8765 and opens the browser
./uninstall.sh      # removes desktop entry/state; keeps project and runs/
```

9.5 Clean-machine test matrix

Test	Check	Pass
Fresh extract	Empty directory; checksums verify	No missing files
No root	Normal user; no `sudo`	Install completes
Offline	Network off; bundled tools	Level 5 demo completes
Loopback	`ss -ltnp | grep 8765`	127.0.0.1 only

Test	Check	Pass
Smoke	Open UI; run Level 5 demo	Comparison + export
Uninstall	`./uninstall.sh`	Desktop entry gone; project intact

9.6 Reviewer experience

1. Download ReverseShield-1.0-fedora.tar.gz and CHECKSUMS.txt.
2. Verify the checksum, extract, and run ./install.sh.
3. Run ./run.sh; the browser opens the local workbench.
4. Click **Run Level 5 demo**; watch phases; inspect the comparison.
5. Export the sanitized report; open the security evidence in docs/security/.

10. Three-day execution plan

The detailed task schedule, playbooks, and checklists are in the companion document. The table below is the summary used in this report.

Day	Focus	Output	Gate
Day 1	Baseline freeze; release artifact hygiene; per-build key + encoded policy table; report sanitizer; one-click skeleton	Release ELF, sanitizer, demo path	Tests pass; no debug/symbols/paths; key differs per build
Day 2	Comparison dashboard; live progress; jobs/storage/export; SAST; ZAP baseline + headers; packaging skeleton	Dashboard, security reports, installer scripts	No high/medium unaddressed; UI size matches disk; no dead buttons
Day 3	Clean Fedora install; end-to-end demo; Word/PDF finalization; rehearsal; submission	Tarball, checksums, two DOCX + PDFs	Fresh install works; demo < 8 min; no placeholders

Priority	Scope	Cut rule
P0	Artifact hygiene, sanitized report, one-click flow, installer, tests	Never cut
P1	Content dashboard, SAST/DAST, jobs/storage, export	Cut only after Day 2 checkpoint
P2	Podman/AppImage, authenticated ZAP API scan, anti-tamper, dynamic binary analysis	Cut first

11. Discussion

The three experiments tell a consistent story. Stripping removes names and debug metadata, but the instructions remain; Ghidra recovered the policy function after the symbol table was gone. Constant blinding changes the instructions and the file, but a naive implementation leaves the key in .data, the debug symbols in place, and most policy constants untouched; Ghidra recovered the same logic and even removed the guard. Binary metadata hygiene removes annotations from any ELF copy, but it also leaves the instructions intact and did not slow Ghidra (6.71 s → 5.92 s on Level 5). In all three cases, the aggregate counts (size, symbols, strings) are a poor proxy for analytical difficulty.

The project's contribution is therefore not 'we protected a binary'. It is 'we built a local workbench that makes the effect of a transformation measurable, and we report the result honestly, including the weaknesses'. That is a

more defensible claim for a student project than a security score, and it produces artifacts a reviewer can reproduce: hashes, commands, tool versions, before/after reports, and a clear limitations section.

The demo protection is intentionally demonstration-grade. The upgrade plan strengthens it in three ways that are visible and testable: artifact hygiene (no debug/symbols/paths), coverage (encode the policy table, not two literals), and key diversity (per-build key). It does not claim resistance; a determined analyst can still recover the key. The value is that the remaining weaknesses are documented and measured rather than hidden.

12. Future work

- **Dynamic binary analysis in a sandbox:** strace/ltrace, gdb, rr, or Frida on self-authored samples, with explicit authorization, snapshots, and no host network.
- **Stronger protection experiments:** table encryption with runtime decode, per-build polymorphism, control-flow flattening, and integrity checks — each with a measurement plan.
- **Anti-tamper and anti-debug:** only as a documented experiment, not a security guarantee; verify behavior with a harness.
- **Cross-platform packaging:** Ubuntu/Kali AppImage or deb, Windows portable build, macOS bundle; keep the static-only boundary.
- **CI and supply chain:** reproducible builds, checksums, SBOM, dependency scanning, signed releases.
- **Multi-user and retention:** authenticated sessions, per-user runs, automatic retention and export policies.
- **Better DAST:** an OpenAPI spec for the local API, authenticated ZAP API scan, and a small fuzzing harness for upload and path handling.
- **Evaluation at scale:** test on larger real applications, multiple toolchains, and measure inter-rater time-to-answer rather than only artifact counts.

13. Conclusion

ReverseShield demonstrates a complete, local, static-only workflow for discovering application targets, running bound analysis tools, observing live progress, and comparing a sample before and after a transformation. The evaluation shows that symbol removal and binary metadata hygiene are metadata changes, not protection, and that a naive constant-blinding pass leaves enough evidence for a decompiler to recover the policy logic. The workbench's value is the measurement discipline: hashes, tool versions, coverage limits, explicit N/A, and limitations that prevent overclaiming.

The three-day plan converts the prototype into a Fedora one-command bundle with a one-click demo, content-level comparison, sanitized reports, and SAST/DAST evidence. If the plan is executed, a reviewer will be able to download the project, run it without root, complete the demo in minutes, and inspect both the evidence and its boundaries.

References

[1] OWASP Foundation. OWASP Web Security Testing Guide. https://owasp.org/projects/web-security-testing-guide

[2] ZAP. Baseline Scan. https://www.zaproxy.org/docs/docker/baseline-scan/

[3] ZAP. API Scan. https://www.zaproxy.org/docs/docker/api-scan/

[4] Semgrep. Supported languages. https://docs.semgrep.dev/supported-languages

[5] Bandit. Getting Started. https://bandit.readthedocs.io/en/latest/start.html

[6] Gitleaks. Secret scanning. https://github.com/gitleaks/gitleaks

[7] TruffleHog. Documentation. https://github.com/trufflesecurity/trufflehog

[8] ProjectDiscovery. Running Nuclei. https://docs.projectdiscovery.io/tools/nuclei/running

[9] GNU Binutils. strip. https://sourceware.org/binutils/docs/binutils/strip.html

[10] Ghidra. Software reverse engineering suite. https://ghidra-sre.org/

[11] OWASP. Top 10 (2021). https://owasp.org/Top10/

[12] MITRE. CWE-200: Exposure of Sensitive Information. https://cwe.mitre.org/data/definitions/200.html

[13] MITRE. CWE-798: Use of Hard-coded Credentials. https://cwe.mitre.org/data/definitions/798.html

[14] MITRE. CWE-312: Cleartext Storage of Sensitive Information. https://cwe.mitre.org/data/definitions/312.html

[15] Podman. Documentation. https://podman.io/docs

[16] AppImage. Documentation. https://docs.appimage.org/

Appendix A — API routes

Method	Route	Purpose
GET	/api/status	Version, tools, mode, limits, token
GET	/api/jobs	Session job list
GET	/api/jobs/<id>?after=N	Events after sequence N
GET	/api/jobs/<id>/report	Download full report
GET	/api/jobs/<id>/artifact	Download stripped/protected artifact
POST	/api/discover	Sample, Flatpak, or local path discovery
POST	/api/ingest	Upload ELF/RPM bytes
POST	/api/jobs	Start analyze / strip_compare / protect_compare
POST	/api/jobs/<id>/cancel	Request cancellation

Appendix B — Run storage

```
runs/<job_id>/
  ├── original-0.elf            # snapshot (0600)
  ├── original-N.elf             # additional targets
  ├── events.jsonl              # live events
  ├── report.json               # final report
  ├── analysis-<hex>/             # analyze job: Ghidra export/logs
  ├── before-<hex>/              # compare: baseline
  ├── after-<hex>/               # compare: reanalysis
  ├── symbols-removed.elf        # strip artifact
  └── protected/
     ├── *-protected.elf
     └── transformed-src/
```

Appendix C — Bundled samples

ID	Name	Complexity	Topics
level-1	Level 1 · Badge Math	Starter	Single function, constant, ELF header
level-2	Level 2 · Tiered Score	Basic	Functions, branches, switch, strings
level-3	Level 3 · Lookup Transform	Intermediate	Lookup table, loop, optimized build

ID	Name	Complexity	Topics
level-4	Level 4 · Record Decoder	Advanced	Structs, bounds checks, parser, checksum
level-5	Level 5 · Policy Engine	Multi-file	Multiple files, enums, structs, interprocedural symbols

Appendix D — Tool versions and commands

Tool	Version / path	Use
file	file-5.46	File identification
readelf	GNU 2.45.1-5.fc43	Headers, sections, symbols
objdump	GNU 2.45.1-5.fc43	Static disassembly
strings	GNU 2.45.1-5.fc43	Printable strings
nm	GNU 2.45.1-5.fc43	Named symbols
Ghidra	12.2 (`tools/ghidra`)	Decompiler, CFG, call edges
JDK	Temurin 25.0.5+8 (`tools/jdk25`)	Ghidra runtime
GCC	15.3.1	Sample and protected build

```
python3 -m unittest discover -s tests
python3 -m py_compile app.py analysis.py workflow.py engine.py \
  discovery.py rpm_ingest.py demo_protection.py http_upgrade.py
node --check static/workbench.js
RUN_GHIDRA_TESTS=1 python3 -m unittest discover -s tests -p 'test_engine.py' -v
```

Appendix E — Report excerpts

Report	Key values
Level 5 strip	16,088 → 10,784 B; sections 36 → 27; strings 132 → 61; functions 13 → 11; BuildID unchanged
Level 5 blinding	16,088 → 16,528 B; `.text` 443 → 581 B; key 0x9E3779B9 twice in `.data`; Ghidra 6.34 s
EasyEffects	11 ELFs; main 13.3 MB / 32 sections / 911 symbols; runtime `KDE6Platform` not scanned
Tests	54 tests, 1 skipped; opt-in Ghidra test passes separately

Appendix F — Security scan commands

```
# SAST
python3 -m venv .venv-security
.venv-security/bin/pip install bandit semgrep detect-secrets
.venv-security/bin/bandit -r app.py analysis.py workflow.py engine.py \
  discovery.py rpm_ingest.py demo_protection.py http_upgrade.py \
  -f json -o docs/security/bandit.json
.venv-security/bin/semgrep scan --config p/security-audit --config p/secrets \
  --config p/python --config p/javascript --metrics=off \
  --json --output docs/security/semgrep.json \
```

```
  app.py analysis.py workflow.py engine.py discovery.py rpm_ingest.py \
  demo_protection.py http_upgrade.py static/workbench.js
.venv-security/bin/detect-secrets scan --all-files \
  --exclude-files 'tools/.*|runs/.*|backups/.*' . \
  > docs/security/detect-secrets.json

# DAST baseline (local only)
docker run --rm --network=host \
  -v "$PWD/docs/security:/zap/wrk:Z" \
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t http://127.0.0.1:8765 \
  -r zap-baseline.html -J zap-baseline.json
```

Appendix G — Glossary

Term	Meaning
ELF	Executable and Linkable Format used by Linux binaries
DWARF	Debug information format; maps instructions to source
Symbol table	Names and addresses for functions, objects, files
Ghidra	NSA open-source reverse-engineering suite with a decompiler
CFG	Control-flow graph: basic blocks and edges
SAST	Static application security testing (source/code review)
DAST	Dynamic application security testing (running HTTP target)
Opaque predicate	A branch whose condition is always true/false by construction
Sanitized report	An export with local paths, secrets, and PII redacted